Cybersecurity Architecture Requirements — Response Document

Project: Equipment Management System **Submitted by:** Myeongjun Kwon, Laboratory Engineer Associate, UL Korea **Date:** 2026-04-02
Reviewer: Lance Morrow, CISSP, Cybersecurity Architect

Q1. What is the project/application name?

Equipment Management System

An internal laboratory equipment management application based on UL-QP-18 (Equipment Management Procedure). It manages equipment lifecycle, calibration records, checkout/return workflows, and approval processes for UL Solutions Korea test laboratories (Suwon, Uiwang, Pyeongtaek).

Q2. Is this a third party solution — SaaS software, application, or vendor service?

No

This is an internally developed application built by UL Korea's laboratory engineering team. The technology stack is:

- **Frontend:** Next.js 16 (React 19, TypeScript)
- **Backend:** NestJS 10 (Node.js 20, TypeScript)
- **Database:** PostgreSQL 15
- **Cache:** Redis
- **File Storage:** S3-compatible object storage (RustFS)
- **All components are containerized** using Docker Compose

No third-party SaaS or vendor services are used for core functionality.

Q3. Is any associated technology hosted in a UL Solutions cloud environment (Azure, AWS, Oracle, Google Cloud)?

UL Solutions Azure tenant (ul.onmicrosoft.com, Entra ID P2) — Azure AD for SSO authentication only.

All other infrastructure is hosted on-premises:

- Application server: Desktop PC in Suwon office secured test lab
- Host OS: **Windows with BitLocker full-disk encryption (AES-256)** + **WSL2 (Ubuntu)**
- Container runtime: **Docker Engine** inside WSL2 (no Docker Desktop — license compliance)
- Database (PostgreSQL): Docker container on the same host
- Cache (Redis): Docker container on the same host
- File storage (RustFS): Docker container on the same host
- Monitoring (Prometheus/Grafana/Loki): Docker containers on the same host
- Reverse proxy (Nginx): Docker container on the same host

Network environment:

- The server PC is located in a secured test lab in Suwon, Korea
- No Wi-Fi module installed — wired LAN connection only

- Connected exclusively to the UL Solutions corporate intranet via LAN cable
- No outbound internet access — blocked by company policy
- Accessible from other UL Korea sites (Uiwang, Pyeongtaek) via the corporate intranet (VPN/MPLS between sites)
- Port assigned by Global Cybersecurity Compliance team

No cloud-hosted compute, database, or storage. Azure AD is used exclusively for user authentication (SSO) via the existing UL Solutions Azure tenant.

Q4. Does this project involve a UL Solutions-developed application or an integration with an internal application?

Yes

- Internally developed by UL Korea laboratory engineering team
- Integrates with UL Solutions Azure AD tenant for SSO authentication
- Uses Azure AD group membership for automatic team/site assignment

---

Q5. Is the application used in a UL Solutions Lab?

Yes

The application is designed specifically for UL Solutions Korea test laboratories:

- **Suwon Lab** (수원): FCC EMC/RF, General EMC, SAR, Automotive EMC teams
- **Uiwang Lab** (의왕): General RF team
- **Pyeongtaek Lab** (평택): Automotive EMC team

It manages laboratory equipment lifecycle per UL-QP-18 procedure requirements.

Q6. Is the application available to the public, customers or otherwise externally accessible?

No

The application is accessible only within the UL Solutions corporate intranet. It is not exposed to the internet or accessible by external parties. Access requires:

1. Connection to the UL Solutions corporate intranet (Suwon, Uiwang, or Pyeongtaek office LAN)
2. Azure AD authentication (SSO)
3. Role-based authorization (5 roles with 72 granular permissions)

The server has no outbound internet access and no Wi-Fi capability. It is connected via LAN cable only, with a port assigned by Global Cybersecurity Compliance.

Q7. The in-scope application is entered into the Application Portfolio and ServiceNow CMDB.

No — Registration pending.

Remediation plan: Will submit ServiceNow CMDB registration request to IT team with the following details:

- Hosting location: On-premises (Suwon office)
- Application owner: Myeongjun Kwon
- Supporting team: UL Korea Laboratory Engineering
- Criticality: Internal operational tool (laboratory equipment management)

Q8. The in-scope application has the latest supported software, library, and firmware updates.

Yes

Component	Version	Status
Node.js	20.x LTS	Current LTS
Next.js	16.x	Latest stable
NestJS	10.x	Current supported
React	19.x	Latest stable
PostgreSQL	15	Current supported
Redis	7 Alpine	Current supported
TypeScript	5.1+	Current

Automated dependency management:

- **Dependabot** configured for weekly npm and GitHub Actions updates (`.github/dependabot.yml`)
- **npm audit** runs in CI pipeline, failing on High/Critical (CVSS 7+) vulnerabilities
- **pnpm overrides** for known vulnerability mitigations (11 packages currently managed)
- **pnpm verify-store-integrity=true** ensures package integrity

---

Q9. The in-scope application has recurring security testing in place.

Yes (Automated — recurring on every push, PR, and weekly schedule)

Automated security testing (CI/CD pipeline — runs on every push/PR):

Gate	Test	Tool	Frequency
Gate 1	Secret scanning	gitleaks v2	Every push/PR
Gate 2	Security decorator validation	Custom `pnpm security:check`	Every push/PR
Gate 2	TypeScript strict mode	`@typescript-eslint/no-explicit-any: error`	Every push/PR
Gate 3	Dependency vulnerability audit	`pnpm audit --prod --audit-level=critical`	Every push/PR
Gate 3	Unit test suite	Jest (25 auth-related tests)	Every push/PR

SAST	Static Application Security Testing	GitHub CodeQL (`security-extended` queries)	Every push/PR + **weekly schedule**

Pre-commit hooks (Husky):

- lint-staged (Prettier formatting on every commit)
- commitlint (conventional commit format enforcement)
- Full ESLint validation runs in CI Gate 2

Dependency monitoring:

- Dependabot weekly scans for npm and GitHub Actions updates

Pending: Formal penetration testing schedule to be established with Global Cybersecurity team.

Q10. The in-scope application has a WAF and/or load balancer in front of the application, if externally-facing.

Other — Not applicable

The application is **not externally facing** (Q6 = No). It is accessible only within the UL Solutions corporate intranet (no internet access, LAN only).

However, the application does employ an **Nginx reverse proxy** with security features:

- Rate limiting: 100 req/min global, 60 req/min API, **10 req/min for auth endpoints**
- Request size limiting: 100MB max body
- Security headers (HSTS, X-Frame-Options, X-Content-Type-Options, etc.)
- TLS 1.2/1.3 termination with NIST-approved cipher suites

Q11. The in-scope application meets identity and access management requirements.

Yes

Authentication

- **Production:** Azure AD SSO (UL Solutions tenant) — local authentication is blocked (`AUTH_PRODUCTION_AZURE_ONLY`)
- **MFA:** Enforced via Azure AD Conditional Access policies
- **Session management:**
  - Access token: 15 minutes
  - Refresh token: 7 days (with rotation)
  - Absolute session maximum: 30 days (forced re-login)
  - Idle timeout: 30 minutes
  - Multi-tab session sync via BroadcastChannel
  - Token blacklist on logout (Redis-backed)

Authorization (Least Privilege)

- **4 roles:** Test Engineer (L1) < Technical Manager (L2) < Quality Manager (L3) < Lab Manager (L4)
- **72 granular permissions** mapped to roles across 16 modules (e.g., `APPROVE_CHECKOUT` , `VIEW_CALIBRATION` , `MANAGE_USERS`)
- **Default-Deny mode:** `PermissionsGuard` denies all access unless explicitly granted via `@RequirePermissions()`

- **Server-side user extraction:** User identity is always extracted from JWT token, never from client request body

Password/Credential Security

- **No hardcoded credentials** in production code — development passwords require explicit environment variables
- **No default passwords** — removed all fallback values from application code
- **Credentials storage:** Environment variables with minimum entropy requirements:
  - JWT_SECRET: 16+ characters (32+ recommended)
  - REFRESH_TOKEN_SECRET: 16+ characters, must differ from JWT_SECRET
  - INTERNAL_API_KEY: 32+ characters
  - All secrets generated via `openssl rand` (384-bit entropy)
- **Timing-safe comparison** for API key validation (`crypto.timingSafeEqual`)

Brute-Force Protection

- Account lockout after 5 failed attempts (15-minute lock duration)
- Auth endpoint rate limiting: 10 requests/minute

Q12. Production data is not available in any environment outside of production.

Yes

- Development environment uses seed data (test fixtures), not production data
- Test environment uses separate test fixtures with UUID constants
- Database connection credentials differ between environments (enforced by env validation — production rejects default password `postgres`)

Q13. Developers are trained in secure coding techniques, including OWASP Top 10.

Other — No formal secure coding training completed, but OWASP Top 10 principles are applied in the codebase.

The developer (sole developer) has not completed a formal OWASP or secure coding training program. However, the application is built with OWASP Top 10 mitigations integrated into the architecture:

OWASP Category	Mitigation
A01: Broken Access Control	RBAC with Default-Deny PermissionsGuard, server-side user extraction
A02: Cryptographic Failures	TLS 1.2/1.3, HMAC-SHA256 JWT signing, NIST-approved ciphers
A03: Injection	Drizzle ORM (parameterized queries), Zod input validation on all endpoints
A04: Insecure Design	CAS/optimistic locking for all state changes, audit logging
A05: Security Misconfiguration	Helmet CSP, CORS whitelisting, env validation with Zod
A06: Vulnerable Components	Dependabot, npm audit in CI, pnpm overrides
A07: Auth Failures	Azure AD SSO, token rotation, absolute session expiry, brute-force protection
A08: Software/Data Integrity	pnpm verify-store-integrity, gitleaks in CI, pre-commit hooks
A09: Security Logging	AuditLog decorator on all state-changing endpoints, event-based logging

A10: SSRF	Internal API key guard with timing-safe validation

Q14. Separation of duties and environments are in place.

Yes

Environment Separation

Environment	Database	Auth Mode	Guard Mode
Development	Local PostgreSQL	Local credentials	DENY
Test	Same DB, test fixtures	Test login endpoint	DENY
Production	Production PostgreSQL	Azure AD only	DENY

Duty Separation (UL-QP-18 Compliance)

- **Test Engineer:** Equipment CRUD, calibration registration
- **Technical Manager:** Approval authority, calibration plan creation
- **Quality Manager:** Calibration plan review (mostly read-only)
- **Lab Manager:** Full authority **except** calibration registration (regulatory duty separation)

Q15. Access to source code and repositories is restricted with least privilege and MFA.

Yes

- Source code hosted on **GitHub** private repository
- Repository access restricted to the sole developer (Myeongjun Kwon)
- GitHub account has **MFA** enabled
- Branch protection rules on `main` branch (CI checks must pass before merge)

Q16. Software applications are developed based on secure coding guidelines and OWASP Top 10.

Yes

See Q13 for detailed OWASP Top 10 mitigation matrix. Additional measures:

- **ESLint rules:** `@typescript-eslint/no-explicit-any: error`, explicit return types enforced
- **Zod validation:** All API endpoints use Zod schema validation (not class-validator)
- **Import restrictions:** ESLint `no-restricted-imports` prevents SSOT violations
- **Security decorator enforcement:** CI script validates all controller endpoints have auth decorators
- **Path traversal prevention:** File storage providers validate all paths against base directory

---

Q17. Source code is reviewed by appropriate personnel.

Yes

- Pull request-based code review workflow
- GitHub branch protection requires review before merge
- Automated CI checks must pass before merge (5-gate pipeline)

Q18. Internally developed code is stored in approved repositories with security functionalities.

Yes

Security Feature	Tool	Status
Secret scanning	gitleaks v2 (CI Gate 1)	Active
Static application security testing (SAST)	GitHub CodeQL (`security-extended`)	Active (every push/PR + weekly)
Dependency vulnerability checks	pnpm audit (CI Gate 3) + Dependabot	Active
Source code analysis	ESLint strict rules + security decorator checker	Active
Pre-commit hooks	Husky + lint-staged	Active
Package integrity	pnpm verify-store-integrity	Active

Q19. Internally developed applications undergo threat modeling and remediation upon each new release.

Yes

Threat modeling process established:

1. **Baseline threat model:** STRIDE-based threat model completed with 35 threats identified and analyzed (see attached `THREAT-MODEL.pdf`)
2. **Per-release threat assessment (PR template):** Every pull request includes a mandatory STRIDE threat assessment checklist:
    - Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege
    - For significant changes (new modules, auth changes, external integrations), each category must be explicitly reviewed
    - PR template: `.github/PULL_REQUEST_TEMPLATE.md`
3. **Release security checklist:** Documented pre-release process (`docs/security/RELEASE-SECURITY-CHECKLIST.md`) covering:
    - Change scope assessment
    - STRIDE threat evaluation per change
    - Automated security verification (CI gates + CodeQL)
    - OWASP Top 10 spot check
    - Remediation sign-off

4. **Automated security gates:** CodeQL SAST analysis runs on every push/PR and weekly, catching common vulnerability patterns automatically

---

Q20. Externally available APIs require proper authentication and authorization.

Yes

Note: APIs are not externally available (internal network only). However, all APIs enforce authentication and authorization:

- **Global JWT guard:** All endpoints require valid JWT token (bypass only via `@Public()` decorator)
- **Permission guard:** All mutation endpoints require specific permissions via `@RequirePermissions()`
- **Internal service guard:** Service-to-service calls use `X-Internal-Api-Key` header with timing-safe validation
- **Rate limiting:** Tiered throttling (global 100/min, API 60/min, auth 10/min)

Q21. API communications are encrypted and only available to trusted, authorized users.

Yes

- **Encryption:** All HTTP traffic over TLS 1.2/1.3 via Nginx reverse proxy
  - NIST-approved cipher suites only (AES-128-GCM, AES-256-GCM, CHACHA20-POLY1305)
  - HSTS enforced (max-age=63072000, includeSubDomains, preload)
  - HTTP→HTTPS redirect
  - OCSP stapling enabled
  - Session tickets disabled (forward secrecy)
- **Authorization:** Bearer token (JWT, HMAC-SHA256) required on all API calls
- **Trusted users only:** Azure AD SSO + RBAC permission model

---

Q22. Does this project involve a UL Solutions-managed server?

Yes

Desktop PC located in a secured test lab in UL Solutions Suwon office.

Server environment:

- **Host OS:** Windows with BitLocker full-disk encryption (AES-256, NIST approved)
- **Container runtime:** WSL2 (Ubuntu) + Docker Engine (no Docker Desktop — enterprise license compliance)
- All application containers run inside WSL2; Docker volumes encrypted at rest via BitLocker

Network characteristics:

- Connected via LAN cable to the corporate intranet (no Wi-Fi module)
- No outbound internet access (blocked by company policy)
- Port assigned by Global Cybersecurity Compliance team
- Accessible from all UL Korea sites (Suwon, Uiwang, Pyeongtaek) via corporate intranet

---

Q23. In-scope servers have the latest supported software, patches and firmware updates.

Yes

Component	Version	Base Image
Docker Host OS	Linux (WSL2/Ubuntu)	Host-managed
Backend	Node.js 20 Alpine	`node:20-alpine`
Frontend	Node.js 20 Alpine	`node:20-alpine`
PostgreSQL	15	`postgres:15`
Redis	7 Alpine	`redis:alpine`
Nginx	Alpine	`nginx:alpine`
Monitoring	Loki 2.9.4, Promtail 2.9.4	Pinned versions

Automated update mechanisms:

- **Dependabot** (weekly): npm dependencies + GitHub Actions
- **pnpm audit** (every CI build): High/Critical vulnerability gate
- **CodeQL SAST** (weekly + every push): Static security analysis
- **Alpine base images**: Minimal attack surface (~5MB, no unnecessary packages)

Q24. In-scope servers have recurring vulnerability scans and a monthly patching cadence.

Yes

Monthly patching cadence defined (see `docs/security/PATCHING-SCHEDULE.md`):

- **Window:** First Monday of each month, 18:00-20:00 KST
- **Scope:** Windows Update, WSL2 kernel, Docker images, application dependencies
- **Vulnerability response SLAs:** Critical 72h, High 2 weeks, Medium next window, Low quarterly

Automated vulnerability scanning (recurring):

Scan Type	Tool	Frequency
Static application security testing	GitHub CodeQL (`security-extended`)	Weekly + every push/PR
Dependency vulnerability audit	pnpm audit (High/Critical gate)	Every CI build
Dependency updates	Dependabot	Weekly
Secret scanning	gitleaks v2	Every push/PR
Manual dependency review	`pnpm audit`	Monthly (during patching window)

Host-level patching:

- Windows Update managed via corporate WSUS/SCCM policy (to be confirmed with IT team)

- Docker image updates: `docker compose pull` during monthly window

Q25. In-scope servers have an approved access control mechanism and least privilege.

Yes

Container-level access control:

- Backend and Frontend containers run as **non-root users** (`USER nestjs` / `USER nextjs` in Dockerfile)
- Docker resource limits enforced (CPU + memory caps per container)
- All application containers isolated on Docker internal networks

Database access control:

- PostgreSQL accessible only on Docker internal network (no published ports)
- Password-protected via `POSTGRES_PASSWORD` environment variable
- Production password validation rejects default values

Cache access control:

- Redis password-protected via `--requirepass` flag
- Accessible only on Docker internal network (no published ports)
- Memory limited to 256MB with LRU eviction policy

Q26. In-scope servers are not directly exposed to the Internet and are placed behind a firewall.

Yes

- **No internet exposure:** The server PC has no outbound internet access (blocked by company policy) and no Wi-Fi module
- **Docker network isolation:**
  - `app-network` is marked `internal: true` — no external access to application containers
  - `monitoring-network` is marked `internal: true`
  - Only Nginx reverse proxy exposes ports 80/443 to the corporate intranet
- **No published database/cache ports:** PostgreSQL (5432), Redis (6379), RustFS (9000) are accessible only within the Docker internal network
- **Physical security:** Server located in a secured test lab with restricted physical access

---

Q27. In-scope servers are hardened — removing/disabling unneeded ports, applications, services and default accounts.

Yes

Port hardening:

- Only 2 ports exposed to network: 80 (HTTP→HTTPS redirect) and 443 (HTTPS)
- All other ports (PostgreSQL 5432, Redis 6379, RustFS 9000, Backend 3001, Frontend 3000, Prometheus 9090, Grafana 3000) are on internal Docker networks only

Service hardening:

- Alpine-based images (minimal packages, no unnecessary services)
- Nginx: `server_tokens off` (version disclosure disabled), custom error pages
- Non-root container execution (backend: `nestjs`, frontend: `nextjs`)

Default account hardening:

- Redis: password-protected, no anonymous access
- PostgreSQL: password-protected, production rejects default password `postgres`

Security headers (Nginx):

- `Strict-Transport-Security`, `X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`
- `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy: camera=(), microphone=(), geolocation=()`

Q28. Does this project involve a UL Solutions-managed database?

Yes

PostgreSQL 15 running as a Docker container on the same server (Windows + WSL2 Ubuntu, BitLocker full-disk encryption).

Q29. Databases are placed behind a firewall, never exposed to the Internet and only accessible from required applications.

Yes

- PostgreSQL runs on Docker internal network (`app-network: internal: true`) — **no published ports**
- Only the Backend (NestJS) container can access PostgreSQL on port 5432
- The server has no internet access — database is completely isolated from external networks
- Direct client access (e.g., pgAdmin from external machines) is not allowed; database management is done via Drizzle ORM CLI from the server

---

Q30. All tables containing sensitive data are encrypted.

Yes

- **Full-disk encryption** via BitLocker on the host OS (Windows) — all Docker volumes including PostgreSQL data are encrypted at rest
- **Transport encryption** supported via `DB_SSL=true` for PostgreSQL connections (opt-in, not needed for Docker internal network)
- The database stores laboratory equipment records, calibration data, and employee names/emails — no financial or healthcare data

---

Q31. Databases are supported by the vendor, patched and backed up regularly.

Yes

- **PostgreSQL 15**: Current vendor-supported version with active security patches
- **Docker image updates**: Manual pull during monthly patching window (`docker compose pull`)
- **Backup**: PostgreSQL data stored in Docker named volume (`postgres_data`), backup via `pg_dump` scheduled

---

Q32. Privileged access to databases is tightly controlled to only authorized administrators.

Yes

- Database password is environment-variable injected (`POSTGRES_PASSWORD`), generated with `openssl rand` (192-bit entropy)
- Production environment validation **rejects the default password** `postgres`
- Only the Backend application container accesses the database — no direct human access in normal operation
- Database administration (migrations, schema changes) is performed by the sole developer via the server terminal

---

Q33. Role/permission assignments follow the principle of least privilege.

Yes

- Application-level RBAC: 5 roles with 72 granular permissions, Default-Deny guard
- Database-level: single application user account with necessary privileges only (no superuser access for the application)
- Server access: restricted to the sole developer/administrator

---

Q34. Direct access to databases only uses accounts specific to a user — never shared accounts.

Yes

- The application connects via a **dedicated application account** (`DB_USER` environment variable)
- No shared database accounts — each environment (dev/test/prod) uses distinct credentials
- The `postgres` superuser exists for initial setup only; application runtime uses the designated user

---

Q35. SA accounts are renamed and disabled where possible; SA passwords are reset from default and properly vaulted.

Other — PostgreSQL does not use the concept of "SA accounts" (that is SQL Server terminology).

The PostgreSQL equivalent (`postgres` superuser) is:

- Password-protected with a strong generated password (`openssl rand` , 192-bit entropy)
- Production environment validation rejects the default password `postgres`
- Used only for initial database setup and migrations; the application connects via a separate user account when configured

---

Q36. Windows Authentication mode is used on SQL servers where possible.

Other — Not applicable.

This project uses **PostgreSQL** (not SQL Server). PostgreSQL runs inside a **Docker container on WSL2 (Ubuntu)**. Windows Authentication mode is a SQL Server-specific feature and does not apply to PostgreSQL.

PostgreSQL authentication uses:

- Password authentication via environment variables
- Optionally SSL client certificates via `DB_SSL=true`

Q37. Vulnerability findings are remediated according to Security Vulnerability Resolution SLAs.

Yes

- **Critical vulnerabilities**: Blocked at CI pipeline (`pnpm audit --prod --audit-level=critical`)
- **Known vulnerabilities**: Mitigated via `pnpm.overrides` (11 packages currently managed)
- **CodeQL SAST**: Runs weekly + on every push/PR, catching vulnerability patterns
- **Dependabot**: Creates automated PRs for vulnerable dependencies weekly

---

Q38. Changes to databases are audited and logged.

Yes

- **Schema changes**: Tracked via Drizzle ORM migration files in source control (Git)
- **Data changes**: All mutation operations logged via `@AuditLog()` decorator with:
  - User ID (server-extracted from JWT)
  - Action type (create/update/delete)
  - Entity type and ID
  - Timestamp
- **Automatic audit**: POST/PATCH/DELETE requests auto-logged by audit interceptor even without explicit `@AuditLog()`

Q39. Non-standard ports are used.

Yes

- PostgreSQL uses **default port 5432** but it is **not exposed outside Docker internal network**
- Redis uses **default port 6379** but similarly **not exposed**
- The only externally accessible ports are **80** (HTTP→HTTPS redirect) and **443** (HTTPS) via Nginx
- The Nginx port on the corporate intranet is **assigned by Global Cybersecurity Compliance** (non-standard)

Q40. Sensitive data, including encryption keys, are encrypted in use and at rest using NIST-approved algorithms.

Yes

Data in Transit

Layer	Encryption	Algorithm	NIST Status
Client ↔ Nginx	TLS 1.2/1.3	AES-256-GCM, CHACHA20-POLY1305	Approved
JWT Signing	HMAC-SHA256	SHA-256	Approved
Key Exchange	ECDHE	P-256, P-384	Approved

Data at Rest

Data	Encryption	Method
All Docker volumes (DB, Redis, Files)	**BitLocker full-disk encryption**	AES-256 (NIST approved)
PostgreSQL data	BitLocker + optional `DB_SSL=true`	AES-256 at disk level
Redis cache	BitLocker + optional `REDIS_TLS=true`	Ephemeral data (TTL-based)
File storage (RustFS)	BitLocker	AES-256 at disk level

Secret Key Management

- All secrets generated with `openssl rand` (192-384 bit entropy)
- Secret rotation supported (dual API key: `INTERNAL_API_KEY` + `INTERNAL_API_KEY_PREVIOUS`)
- **Azure AD client secret:** Managed via environment variable (`AZURE_AD_CLIENT_SECRET`), rotation per Azure Entra ID security baseline (recommended every 60 days)
- Production environment validation rejects weak or default secrets
- Secrets stored as environment variables with `chmod 600` file permissions

---

Q41. Sensitive data is stored securely with appropriate access controls and least privilege.

Yes

- **RBAC:** 5 roles with 72 granular permissions, Default-Deny guard mode
- **Server-side user extraction:** User ID extracted from JWT, never from client input
- **Audit logging:** All state-changing operations logged with `@AuditLog()` decorator
- **CAS (Compare-and-Swap):** Optimistic locking prevents concurrent modification conflicts
- **Token security:** Access tokens expire in 15 minutes, refresh tokens in 7 days, absolute session max 30 days
- **File access:** Presigned URLs with 1-hour expiration for file downloads

---

Q42. Certificates are approved by Global Cybersecurity and signed by a known, trusted provider.

No — Certificate procurement pending.

Current plan:

- Internal network deployment — considering internal CA or commercially signed certificate
- Will coordinate with Global Cybersecurity for certificate approval before production deployment
- Nginx is pre-configured for TLS termination with certificate auto-renewal support

Remediation: Will submit certificate request to Global Cybersecurity team.

Q43. Wildcard certificates are not used in production environments.

Yes

No wildcard certificates are used or planned. The application uses a single domain-specific certificate.

Q44. Does this project involve introducing new networking devices?

No

The application runs on an existing desktop PC connected to the existing corporate intranet via LAN cable. No new firewalls, switches, routers, or wireless access points are introduced. The server PC has no Wi-Fi module and no internet access.

Q45. Architecture diagram upload.

The architecture diagram is attached as a separate PDF file (`ARCHITECTURE-DIAGRAM.pdf`), including:

- System architecture overview (all Docker containers and network topology)
- Network topology (internal/external network boundaries, published ports)
- Authentication flow (Azure AD SSO → NextAuth → NestJS JWT, with protocols: OAuth 2.0, HTTPS, HMAC-SHA256)
- Data flow diagram (PostgreSQL wire protocol, Redis protocol, S3 API, all on Docker internal network)
- RBAC permission model (5 roles, 72 permissions, Default-Deny guard)

Appendix A: Architecture Diagram

See attached `ARCHITECTURE-DIAGRAM.pdf`

Appendix B: Security Implementation Details

See attached `SECURITY-IMPLEMENTATION.pdf`

Appendix C: Threat Model

See attached `THREAT-MODEL.pdf`